TripNest AI: API Documentation

Base URL: the same origin as the web app (local: `http://localhost:8000`). Interactive documentation generated by FastAPI: `/docs` (Swagger UI) and `/openapi.json`. All endpoints are `GET`, need no authentication or API key, and return JSON. Errors use `{ "detail": "message" }` with HTTP 404 (not found) or 422 (invalid input).

Only the two core features and their supporting data are exposed. Itinerary, community, review, weather, account and admin endpoints are **not mounted** (the code is kept, unmounted).

Endpoints

Method	Path	Purpose	Parameters
GET	`/api/attractions/{slug}/forecast`	Attraction Forecast	`slug`*, `date`
GET	`/api/attractions/{slug}/history`	Attraction History	`slug`*
GET	`/api/attractions/{slug}/photo`	Attraction Photo	`slug`*
GET	`/api/categories`	Categories	-
GET	`/api/crowd/compare`	Compare	`slugs`*, `date`
GET	`/api/crowd/highlights`	Highlights	`date`, `limit`
GET	`/api/crowd/map`	Crowd Map	`at`, `destination`
GET	`/api/destinations`	Destinations	`q`, `state`, `category`, `budget`, `month`, `featured`, `sort`, `page`, `limit`
GET	`/api/destinations/{slug}`	Destination	`slug`*
GET	`/api/destinations/{slug}/crowd`	Destination Crowd	`slug`*, `at`
GET	`/api/destinations/{slug}/photo`	Destination Photo	`slug`*
GET	`/api/destinations/{slug}/prediction`	Predicted crowd for a destination, date and time range	`slug`, *`date`*, `from`, `to`
GET	`/api/health`	Health	-
GET	`/api/locations/all`	All cities available as starting points	-
GET	`/api/locations/resolve`	Resolve typed text to a city (manual entry)	`q`*
GET	`/api/locations/search`	Search Indian cities by name	`q`*
GET	`/api/model`	Model Card	-
GET	`/api/recommendations`	Recommended destinations for a start, date and optional interest	`date`*, `lat`, `lon`, `interest`, `max_km`, `limit`
GET	`/api/states`	States	-

`*` = required. Dates are `YYYY-MM-DD` within about 18 months ahead (and up to a year back). Hours are 0–23.

Core feature 1: crowd prediction

`GET /api/destinations/{slug}/prediction?date=2026-12-26&from=9&to=17`

Predicted crowd for a destination, date and time range (defaults `from=8`, `to=18`). `from` must not be after `to`. Response (abridged, real output):


```
{
 "date": "2026-12-26",
 "selected": {
  "from": 9,
  "to": 17,
  "level": "At Capacity",
  "peak_hour": 17,
  "peak_label": "5 PM",
  "visitors": 12216,
  "low": 10839,
  "high": 13765,
  "util": 0.962,
  "avg_util": 0.416
 },
 "best_time": {
  "hour": 11,
  "label": "11 AM",
  "level": "Low",
  "util": 0.151,
  "visitors": 2003
 },
 "hourly": [
  {
   "hour": 5,
   "label": "5 AM",
   "util": 0.339,
   "visitors": 3727,
   "low": 3294,
   "high": 4215,
   "level": "Low"
  },
  {
   "hour": 6,
   "label": "6 AM",
   "util": 0.506,
   "visitors": 5568,
   "low": 4932,
   "high": 6283,
   "level": "Moderate"
  }
 ],
 "week": [
  {
   "date": "2026-12-26",
   "weekday": "Sat",
   "peak_level": "At Capacity",
   "peak_util": 0.962,
   "peak_hour": 17,
   "quietest_label": "11 AM",
   "quietest_level": "Low"
  },
  {
   "date": "2026-12-27",
   "weekday": "Sun",
```

```
    "peak_level": "High",
    "peak_util": 0.897,
    "peak_hour": 17,
    "quietest_label": "11 AM",
    "quietest_level": "Low"
   }
  ],
  "provenance": {
   "type": "model-predicted",
   "training_data": "demonstration",
   "model_trained_at": "2026-11-01T22:54:13",
   "uncertainty": {
    "method": "80% range from the model's error on a held-out test fortnight (log-normal residuals)",
    "confidence": 0.8
   }
  }
}
```


`selected.level` is the level of the busiest hour in the range. `selected.low/high` are an 80% likely range. `best_time` is the quietest daytime hour (7 AM to 7 PM). `week` has the next seven days. `provenance.limitations` lists what the prediction cannot do.

Core feature 2: recommendation

`GET /api/recommendations?lat=13.0827&lon=80.2707&date=2026-12-26&interest=beach&limit=2`

Ranks destinations. `lat` and `lon` come together or not at all; without them distance is not used. `interest` is optional. `max_km` limits distance. `limit` is 1–60 (default 6). Response (abridged, real output):


```
{
 "date": "2026-12-26",
 "weights": {
  "crowd": 0.4,
  "interest": 0.25,
  "distance": 0.2,
  "season": 0.15
 },
 "items": [
  {
   "slug": "mahabalipuram",
   "name": "Mahabalipuram",
   "state": "Tamil Nadu",
   "state_code": "TN",
   "lat": 12.6208,
   "lon": 80.1945,
   "categories": [
    "heritage",
    "beach"
   ],
   "daily_cost": 2200,
   "score": 0.702,
   "breakdown": {
    "crowd": 0.266,
    "interest": 1.0,
    "distance": 0.979,
    "season": 1.0
   },
```

```
   "reasons": [
    "Predicted Moderate crowds on Sat 26 Dec: about 3,765 people at the busiest hour (4 PM), and
quieter at 7 AM",
    "A beach destination",
    "About 52 km from you (straight line)",
    "December is a good time to visit"
   ],
   "cautions": [],
   "distance_km": 52,
   "distance_note": "straight-line distance",
   "in_season": true,
   "crowd": {
    "level": "Moderate",
    "util": 0.661,
    "visitors": 3765,
    "low": 3320,
    "high": 4265,
    "peak_hour": 16,
    "peak_label": "4 PM"
   },
   "suggested_time": {
    "hour": 7,
    "label": "7 AM",
    "level": "Low",
    "util": 0.241
   }
  }
 ]
}
```


`score` = sum of `weights[k] × breakdown[k]`. Weights are re-normalised to total 1 when `interest` or the start location is missing (that factor's weight is then 0). Popularity is not a factor.

Locations

`GET /api/locations/search?q=chen` returns up to 8 matching cities. `GET /api/locations/resolve?q=Bangalore` resolves typed text (aliases and ", State" suffixes are understood):


```
{"name": "Bengaluru", "state": "Karnataka", "lat": 12.9716, "lon": 77.5946}
```


Unknown text returns 404 with a message asking for a nearby major city. `GET /api/locations/all` lists all 77 cities.

Explore (the location-first screen)

`GET /api/explore?lat=11.0168&lon=76.9558&date=2026-12-26&interest=wildlife&slot=all`

`lat`, `lon` and `date` are required; `interest` is optional (letters only, 30 characters); `slot` is `all`, `early`, `morning`, `afternoon` or `evening`. Returns `origin` (the nearest listed city, its `distance_km`, `covered` and a `label` for the greeting), `popular` (catalogue popularity at least 0.9, nearest first; `popular_expanded` and `popular_note` say if the radius had to widen), `hidden_gems` (popularity at most 0.8, predicted Low or Moderate; each with `reasons`, `classification_reason`, `score`, `breakdown` and, where one exists, `alternative_to`), `map` points (`group`, `level`, `color`, `blurb`, `image_url`, `best_label`), `classification` thresholds, `method` and `provenance`. Each card carries `image` (URL, credit, licence), straight-line `distance_km`, `crowd`, `best_window` and `timeline`; `best_window` is null when the data cannot support a recommendation. Invalid coordinates or dates return 422.

`GET /api/locations/nearest?lat=..&lon=..`

Names the closest listed city for a browser position, used for the greeting ("around X", or "near X" when it is far). The response echoes the position rounded to 2 decimals (`query_lat`, `query_lon`) and its `distance_km` from the city.

Supporting endpoints

- `GET /api/destinations?q=&state=&category=&budget=&month=&page=&limit=` lists destinations. `GET /api/destinations/{slug}` gives one with its attractions.
- `GET /api/crowd/compare?slugs=goa,jaipur&date=2026-12-26` compares up to 5 destinations hour by hour.
- `GET /api/crowd/map?at=2026-12-26T12:00` gives every attraction's predicted level and a map `heat` value (0 for Low and Closed, so quiet places stay uncoloured).
- `GET /api/destinations/{slug}/photo` and `GET /api/attractions/{slug}/photo` return a Wikimedia Commons photo with credit and licence, or `available: false` with a reason.
- `GET /api/model` returns the model card (split dates, row counts, metrics, baselines). `GET /api/health` returns service, database and model status.

Conventions

- No API key or login is needed. Browser state-changing requests are not used (all endpoints are read-only).
- Every prediction response carries `provenance`: `type: model-predicted`, `training_data: demonstration`, `generated_at`, `model_trained_at`.
- Responses never contain private data; no personal data is stored.